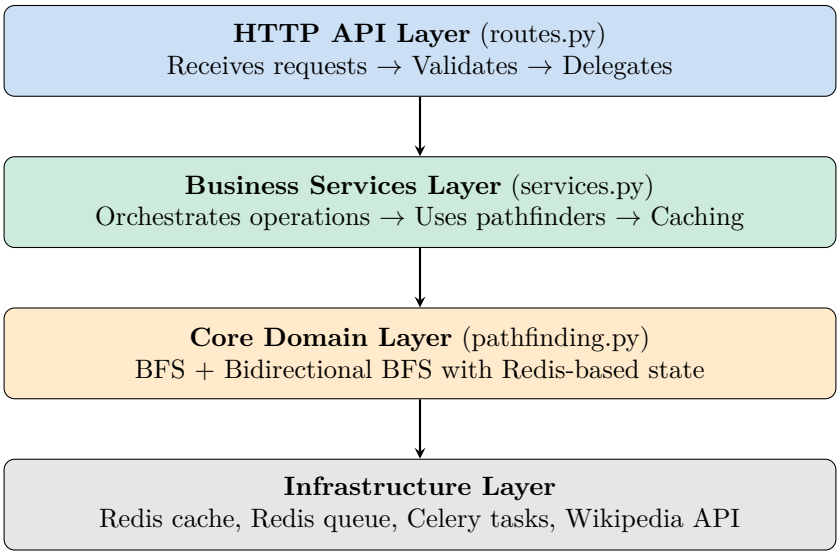
Iris Backend Architecture Guide

Wikipedia Pathfinding Engine



Interview Preparation & Development Reference

Last Updated: March 2026

Contents

1 Project Structure Overview **3**

2 Architecture Layers **3**
2.1 Layer Descriptions . . . 4
2.2 Data Flow Diagram . . . 4

3 Engineering Architecture Decisions **4**
3.1 High-Level Architecture Decisions . . . 4
3.2 Low-Level Implementation Decisions . . . 5
3.3 Scalability Characteristics Summary . . . 8

4 Service Lifecycle & Initialization **8**
4.1 The "When": Lazy Initialization . . . 8
4.2 Component Lifecycle Table . . . 9
4.3 Initialization Flow . . . 9

5 Progress Callback System **9**
5.1 The Problem . . . 9
5.2 The Solution: Callback Injection . . . 9
5.3 Code Flow . . . 10
5.3.1 Step 1: Task Defines the Callback . . . 10
5.3.2 Step 2: Factory Passes Callback to PathFinder . . . 10
5.3.3 Step 3: BFS Defines an `on_page_fetched` Closure . . . 10
5.3.4 Step 4: Wikipedia Client Fires the Closure via `as_completed` . . . 11
5.3.5 Step 4b: Link-Processing Callbacks Close the UI Gap . . . 12
5.3.6 Path Reconstruction via Parent Hash . . . 13
5.3.7 Step 5: Client Sees Progress via Polling . . . 13
5.4 Bidirectional BFS Progress: BidirProgressAggregator . . . 13
5.5 Why This Design? . . . 14

6 Dependency Injection (DI) Explained **14**
6.1 The Pattern: Singleton + Factory . . . 14
6.2 Swapping Implementations . . . 15

7 File-by-File Breakdown **15**
7.1 Core Files . . . 15
7.2 Infrastructure Files . . . 16

8 Caching Strategy **16**

9 Testing Architecture **17**
9.1 Directory Structure . . . 17
9.2 Mock Fixture Design . . . 18
9.3 Testing Celery Tasks . . . 18
9.4 Coverage . . . 18

10 Interview Q&A Cheat Sheet **18**

11 Quick Reference: Where to Make Changes **21**

12 API Endpoints Reference **21**

13 Configuration Reference **22**

1 Project Structure Overview

The Iris backend follows a **Clean Architecture** pattern with clear separation of concerns.

```
iris-web-backend/
|-- app/                          # Main application package
|   |-- __init__.py              # Flask app factory & Celery setup
|   |-- api/                     # HTTP API layer
|   |   |-- routes.py            # API endpoints (Flask blueprints)
|   |   |-- middleware.py        # Decorators: error handling, logging, CORS
|   |   |-- schemas.py           # Marshmallow schemas for validation
|   |   |-- swagger.py           # Flasgger config + API metadata (paths from docstrings)
|   |-- core/                    # Core business logic (THE BRAIN)
|   |   |-- factory.py           # Dependency Injection (ServiceFactory)
|   |   |-- interfaces.py        # Abstract interfaces (contracts)
|   |   |-- models.py            # Data models (dataclasses)
|   |   |-- pathfinding.py       # CORE ALGORITHM (BFS + Bidirectional BFS)
|   |   |-- services.py          # Business services orchestration
|   |-- external/                # External API integrations
|   |   |-- wikipedia.py         # Wikipedia API client
|   |-- infrastructure/          # Infrastructure concerns
|   |   |-- cache.py             # Redis cache implementation
|   |   |-- redis_queue.py       # Redis queue for BFS state
|   |   |-- tasks.py             # Celery background tasks
|   |-- utils/                   # Utilities
|       |-- exceptions.py        # Custom exception hierarchy
|       |-- logging.py           # Logging configuration
|-- config/                      # Configuration classes
|-- tests/                       # Test suites
|   |-- conftest.py              # Shared fixtures (stateful mocks)
|   |-- unit/                    # Unit tests (per-class)
|   |-- integration/             # Integration tests (service interactions)
|-- run.py                       # Application entry point
|-- celery_worker.py             # Celery worker entry point
```

Listing 1: Project Directory Structure

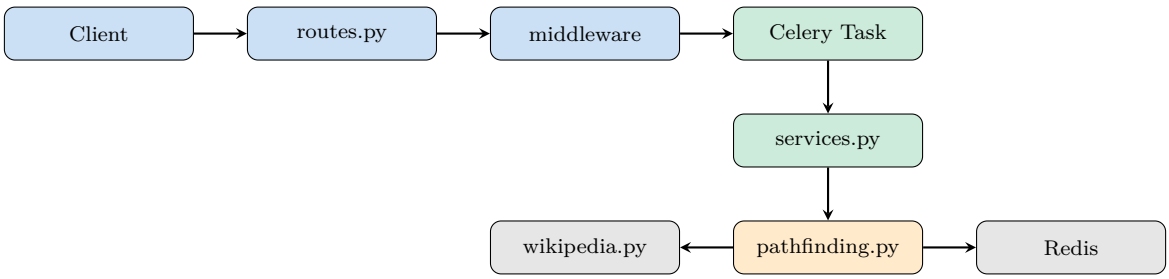
2 Architecture Layers

The codebase follows a **layered architecture** similar to Clean Architecture. Each layer has a single responsibility.

2.1 Layer Descriptions

Layer		Responsibility
HTTP API Layer	`app/api/`	Receives HTTP requests, validates input using Marshmallow schemas, applies cross-cutting middleware (CORS, logging, error handling), delegates to business services, and formats JSON responses. Swagger spec is auto-assembled from YAML docstrings on each route function via Flasgger.
Business Services	`app/core/services.py`	Orchestrates business operations. Checks cache before invoking pathfinding, coordinates between pathfinders and Wikipedia client, encapsulates use cases like "find path" or "get page info."
Core Domain	`app/core/pathfinding.py`	Contains the BFS and Bidirectional BFS algorithms. No knowledge of HTTP or Flask. Only knows about abstract interfaces. This is where the graph traversal logic lives.
Infrastructure	`app/infrastructure/`	Concrete implementations: Redis cache, Redis queue for BFS state, Celery tasks for background processing. Also `app/external/wikipedia.py` for the Wikipedia API client. All "how" details live here.

2.2 Data Flow Diagram



3 Engineering Architecture Decisions

This section justifies key technical decisions from a **distributed systems** and **scalability** perspective. These are the answers that demonstrate engineering depth.

3.1 High-Level Architecture Decisions

Decision: Asynchronous Task Processing with Celery

Problem: Wikipedia pathfinding can take 30 seconds to 5+ minutes. HTTP requests would timeout.
Decision: Offload pathfinding to Celery background workers. API returns a task ID immediately; client polls for results.
Why This Scales:

- **Horizontal Scaling:** Spin up N Celery workers to handle N concurrent searches.
- **Decoupling:** Web servers handle HTTP; workers handle compute. Each scales independently.

- **Resilience:** If a worker crashes, the task stays in Redis and another worker picks it up.

Trade-off: Added complexity of polling. Considered WebSockets but polling is simpler to implement and debug.

Decision: Redis as Cache

Problem: Need caching, a job queue, and result storage. Using 3 different systems increases operational overhead.
Decision: Use Redis for all three: Wikipedia link cache, Celery broker, and Celery result backend.
Why This Scales:
- **Single Point of Expertise:** Operations team only needs to master one system.
- **Redis Cluster:** Can scale to hundreds of thousands of operations/second.
- **Memory Efficiency:** Redis is optimized for this exact use case.

Trade-off: Redis is a single point of failure. Mitigated with Redis Sentinel or Redis Cluster in production.

Decision: Stateless Web Servers

Problem: Need to scale web tier horizontally behind a load balancer.
Decision: All application state lives in Redis. Flask servers are completely stateless.
Why This Scales:
- **Instant Horizontal Scaling:** Add/remove web servers without session migration.
- **Zero Downtime Deploys:** Rolling restarts don't lose user state.
- **Container-Friendly:** Perfect for Kubernetes/Docker deployments.

Decision: Separation of API and Worker Deployments

Problem: API servers need to respond quickly; workers need CPU for graph traversal.
Decision: Deploy API servers and Celery workers as separate services with different resource profiles.
Why This Scales:
- **Independent Scaling:** Scale workers based on queue depth; scale API based on request rate.
- **Resource Isolation:** Workers can use 100% CPU without affecting API latency.
- **Cost Optimization:** Use different instance types (API: network-optimized; Workers: compute-optimized).

3.2 Low-Level Implementation Decisions

Decision: Redis-Based BFS Queue with Batch Node Processing

Problem: BFS at depth 5+ can involve millions of nodes. Python's `deque` would exhaust worker memory. Fetching links one node at a time also wastes Wikipedia API capacity.
Decision: Store BFS queue and visited set in Redis. Each iteration pops a **batch** of nodes (`BFS_BATCH_SIZE`, default 50) and fetches all their links in a single `get_links_bulk` call which fans out to N parallel Wikipedia requests.
Engineering Justification:
- **Memory Safety:** Worker memory stays constant regardless of search depth.
- **Throughput:** Popping 50 nodes and dispatching 50 parallel Wikipedia API calls replaces 50 sequential round-trips per BFS level.

- **Observability:** Can inspect queue depth via Redis CLI for debugging.
- **Crash Recovery:** If worker dies, search state persists in Redis (though we restart fresh currently).

Trade-off: Each Redis `pop_batch` is a network hop, but it is amortised across all nodes in the batch. The net effect is far fewer total round-trips versus per-node fetching.

Decision: Connection Pooling via Singleton Pattern

Problem: Creating new TCP connections to Redis/Wikipedia on every request is expensive.
Decision: Use lazy-loaded singletons for `RedisClient` and `WikipediaClient`.
Engineering Justification:

- **Redis:** Uses `redis.ConnectionPool` – amortizes TCP handshake across requests.
- **Wikipedia:** Uses `requests.Session` – enables HTTP Keep-Alive and connection reuse.
- **TLS Overhead:** Wikipedia uses HTTPS. Reusing connections avoids repeated TLS handshakes (100-300ms saved per request).

Decision: Wikipedia API Fetching: One Request Per Page

Problem: Wikipedia's `prop=links` API returns at most 500 links per request (`pllimit=max`). This cap is **global per API call**, not per title. Grouping multiple titles in one request causes link starvation: a high-link-count hub page (e.g. *Microsoft* with 500+ links) consumes the entire 500-link quota, leaving other pages in the same request with 0 links returned.
Decision: Each page title is fetched in its own API call. Parallelism is preserved via `ThreadPoolExecutor` (`WIKIPEDIA_MAX_WORKERS`, default: 10), so a BFS batch of 50 nodes still issues all 50 requests concurrently. Pagination is followed up to `WIKIPEDIA_MAX_PAGINATE_CALLS` (default: 3) pages per title, yielding up to $3 \times 500 = 1{,}500$ links per page before truncating.
Engineering Justification:

- **Correctness:** Every page receives its full link allocation regardless of what other pages are in the same BFS batch.
- **I/O Bound:** Wikipedia API calls are network-bound; threads maximise throughput without the starvation risk of multi-title requests.
- **Rate Limiting:** Thread pool size implicitly caps concurrent requests to Wikipedia. A shared `threading.Lock`-guarded rate limiter enforces a minimum interval between all requests across threads.

Backoff Strategy: `_request_with_backoff` retries up to `max_retries` (default: 5) times with exponential backoff ($2^{attempt}$ seconds). HTTP 429 responses honour the `Retry-After` header. 5xx server errors trigger the same backoff. Other 4xx errors raise `WikipediaAPIError` immediately without retry.

Decision: Bidirectional BFS using Wikipedia Backlinks

Idea: Standard BFS expands one frontier from the start page. Bidirectional BFS expands **two** frontiers simultaneously — one forward from the start and one backward from the end — and stops when they meet. Theoretical speedup: $O(b^{d/2})$ vs $O(b^d)$.
How the reverse frontier works in Wikipedia: The backward frontier needs to know "which pages link *to* the current target?" This is Wikipedia's `prop=linkshere` API ("what links here"). The `get_backlinks_bulk` method calls this API, caching results under the `wiki_backlinks:` key prefix.
Algorithm (`BidirectionalBFSPathFinder`):

1. Initialise `fwd_queue` with `start_page` (depth 0) and `bwd_queue` with `end_page` (depth 0).
2. Each iteration: expand whichever frontier is **smaller** (load balancing). Forward expands by following outgoing links; backward expands by following backlinks.
3. Visited sets (`fwd_visited`, `bwd_visited`) are Redis sets. Batch membership checks use `set_contains_many` (one pipeline round-trip).
4. When a newly discovered node is already in the *opposite* frontier's visited set, that node is a **meeting point**.
5. If multiple meeting points are found in one batch, `_pick_shortest` selects the one whose combined forward+backward path length is shortest.
6. Path reconstruction: `_reconstruct_path(meeting, fwd_par)` walks forward parent hashes from meeting back to start. `_reconstruct_bwd_chain(meeting, bwd_par)` walks backward parent hashes from meeting forward to end. The two segments are concatenated.

Depth budget: `fwd_max = (max_depth + 1) // 2`, `bwd_max = max_depth // 2`. Each frontier stops expanding at its budget.

Trade-off: Backlinks for very popular pages (e.g. "United States") can return extremely large result sets, making the backward frontier expensive. The "expand smaller frontier" heuristic mitigates this by deprioritising large frontiers. In the worst case (highly asymmetric graph), standard BFS may still be faster; in practice the algorithm often halves search depth.

Decision: BidirProgressAggregator: Thread-Safe Dual-Frontier Progress

Problem: Two frontiers run concurrently (both call `get_links_bulk` / `get_backlinks_bulk` which fire `on_page_fetched` from worker threads). A single progress callback must receive unified stats without race conditions.

Decision: `BidirProgressAggregator` owns all shared counters under one `threading.Lock`. Each frontier calls `aggregator.record(title, depth, direction)` from its `on_page_fetched` closure. The aggregator increments the appropriate counter, reads both queue sizes in one call, and fires the real callback with a unified snapshot.

What it tracks:

- `_fwd_nodes`, `_bwd_nodes` — separate node counts per direction.
- `total_nodes` property — their sum.
- `queue_size` in the callback — sum of both frontier queue lengths.
- `direction` field — lets the UI indicate which frontier found the node.

Decision: Cache TTLs Tuned to Data Volatility

Problem: Wikipedia pages change, but not frequently. Need to balance freshness vs. cache hit rate.

Decision: Different TTLs for different data types.

Engineering Justification:

- `wiki_links:*` and `wiki_backlinks:*` – 24 hours. Page links rarely change; high reuse value.
- `path:*:*` – 1 hour. Paths are computed results; shorter TTL ensures freshness.
- `bfs_*` – explicit cleanup after search completes (not TTL-based). Periodic maintenance task sweeps orphaned keys.

Decision: UUID-Based Session Isolation for BFS State

Problem: Multiple concurrent searches must not interfere with each other's BFS state.
Decision: Each search generates a UUID. All Redis keys are prefixed: `bfs_queue:<uuid>`, `bfs_visited:<uuid>`, `bfs_parent:<uuid>`.
Engineering Justification:

- **No Locking Required:** Each search operates on its own keyspace.
- **Easy Cleanup:** Pattern-based deletion via `delete_many` in a `finally` block — guaranteed to run even if search fails.
- **Debugging:** Can trace a specific search by its UUID in logs and Redis.

Decision: Dependency Injection via Factory Pattern

Problem: Need testable code without complex DI frameworks.
Decision: Hand-rolled `ServiceFactory` with constructor injection.
Engineering Justification:

- **Testability:** Tests inject mocks directly via constructor.
- **No Magic:** Unlike `@inject` decorators, the wiring is explicit and traceable.
- **Cleanup Method:** `ServiceFactory.cleanup()` resets singletons between tests.

3.3 Scalability Characteristics Summary

Dimension	How It Scales
Concurrent Searches	Add more Celery workers (horizontal).
API Request Rate	Add more Flask/Gunicorn instances behind load balancer.
Cache Size	Redis Cluster with sharding or increase instance memory.
Wikipedia Rate Limits	Configurable thread pool; request delay; exponential backoff.
Storage (Results)	Redis with TTL-based eviction; results auto-expire after 1 hour.

4 Service Lifecycle & Initialization

Understanding precisely **when**, **where**, and **how** services are initialized demonstrates production awareness.

4.1 The "When": Lazy Initialization

Services are **not** initialized when you run `python run.py`. They are created **on first use**.

- **Flask Process:** First API request triggers service initialization.
- **Celery Worker:** First task pickup triggers service initialization.

4.2 Component Lifecycle Table

Component	Lifecycle	Reasoning
Redis Client	Singleton	Maintains `ConnectionPool`; avoids TCP handshake overhead per request.
Wikipedia Client	Singleton	Holds `requests.Session`; enables HTTP Keep-Alive and TLS session reuse. Also owns the rate-limiter lock and last-request timestamp.
Cache Service	Singleton	Shared serialization logic wrapping the Redis client.
Queue Service	Singleton	Shared FIFO queue wrapping the same Redis client.
Pathfinding Service	Transient	New instance per task. Allows request-specific `progress_callback` and algorithm choice.

4.3 Initialization Flow

When a Celery task starts:

1. `find_path_task` starts in `infrastructure/tasks.py`.
2. Calls `get_pathfinding_service(algorithm, progress_callback)`.
3. `ServiceFactory` checks `_wikipedia_client`. It's `None` → creates it.
4. To create Wikipedia client, needs `_cache_service`. It's `None` → creates it.
5. To create cache, needs `_redis_client`. It's `None` → creates it.
6. Dependencies wired together via constructor injection; `PathFindingService` returned.
7. **Next task:** Singletons already exist → instant return.

Important

Singletons are Per-Process. If Gunicorn runs 4 workers, there are 4 independent Wikipedia Clients. They share state via the common Redis database, not via shared memory.

5 Progress Callback System

The progress callback allows clients to see **live search progress** rather than waiting blindly.

5.1 The Problem

A BFS search can take minutes. Without progress updates, the client has no idea:

- Is the search still running or stuck?
- How deep has it gone?
- How many nodes has it explored?

5.2 The Solution: Callback Injection

1. **Task creates a callback function** that updates Celery task state.
2. **Callback is injected** into `PathFindingService` via constructor.
3. **BFS passes an `on_page_fetched` closure** into `get_links_bulk`.
4. **Wikipedia client fires the closure** from worker threads as each API response arrives (and also for cache hits), spreading updates across the fetch window.
5. **Client polls `/tasks/status/<id>`** and sees real-time progress.

5.3 Code Flow

5.3.1 Step 1: Task Defines the Callback

```python
@celery.task(bind=True)
def find_path_task(self, start_page, end_page, algorithm="bidirectional"):
    max_depth = current_app.config.get("MAX_SEARCH_DEPTH", 6)

    def progress_update(progress_data):
        # Normalise: pathfinders may emit a flat dict or one with search_stats
        if "search_stats" not in progress_data:
            progress_data = {
                "status": "Searching...",
                "search_stats": progress_data,
            }
        progress_data["search_stats"].setdefault("start_page", start_page)
        progress_data["search_stats"].setdefault("end_page", end_page)
        progress_data["search_stats"].setdefault("max_depth", max_depth)
        self.update_state(state="PROGRESS", meta=progress_data)

    pathfinding_service = get_pathfinding_service(algorithm, progress_update)
    result = pathfinding_service.find_path(search_request)
```

Listing 2: tasks.py – Creating the Progress Callback

5.3.2 Step 2: Factory Passes Callback to PathFinder

```python
@classmethod
def create_pathfinding_service(cls, algorithm="bidirectional", progress_callback=
    None):
    wikipedia_client = cls.get_wikipedia_client()
    cache_service = cls.get_cache_service()
    queue_service = cls.get_queue_service()
    max_depth = current_app.config.get("MAX_SEARCH_DEPTH", 6)
    batch_size = current_app.config.get("BFS_BATCH_SIZE", 50)

    if algorithm.lower() == "bidirectional":
        path_finder = BidirectionalBFSPathFinder(
            wikipedia_client, cache_service, queue_service,
            max_depth, batch_size, progress_callback
        )
    else:
        path_finder = RedisBasedBFSPathFinder(
            wikipedia_client, cache_service, queue_service,
            max_depth, batch_size, progress_callback
        )

    return PathFindingService(path_finder, cache_service, wikipedia_client)
```

Listing 3: factory.py – Injecting the Callback

5.3.3 Step 3: BFS Defines an `on_page_fetched` Closure

```python
nodes_explored = 0
nodes_lock = threading.Lock()   # guards concurrent increments from thread pool

while self.queue_service.length(queue_key) > 0:
    batch_items = self.queue_service.pop_batch(queue_key, self.batch_size)
    current_depth = batch_items[0]["depth"]

    on_page_fetched: Callable | None = None
    if self.progress_callback:
        def _on_page_fetched(title, _links, *, _d=current_depth):
```

```
            # Default arg binds current_depth by VALUE at def time (avoids B023 late-binding).
            # Called concurrently from ThreadPoolExecutor threads.
            nonlocal nodes_explored
            with nodes_lock:
                nodes_explored += 1
                count = nodes_explored
            self.progress_callback({
                "status": "Searching...",
                "search_stats": {
                    "nodes_explored": count,
                    "current_depth": _d,
                    "last_node": title,
                    "queue_size": self.queue_service.length(queue_key),
                },
                "search_time_elapsed": round(time.time() - search_start_time, 2),
            })
        on_page_fetched = _on_page_fetched

    links_bulk = self.wikipedia_client.get_links_bulk(page_names, on_page_fetched)
    # When no progress callback: increment counter here instead of in the closure.
    if on_page_fetched is None:
        nodes_explored += len(batch_items)
```

Listing 4: pathfinding.py – on_page_fetched Closure (RedisBasedBFS)

5.3.4 Step 4: Wikipedia Client Fires the Closure via as_completed

```
def _bulk_fetch(self, page_titles, cache_prefix, fetch_fn, on_page_fetched):
    results = {}
    uncached_titles = []

    # Cache hits fire the callback immediately
    if self.cache_service:
        for title in page_titles:
            cached = self.cache_service.get(f"{cache_prefix}:{title}")
            if cached is not None:
                results[title] = cached
                if on_page_fetched:
                    on_page_fetched(title, cached)   # fires for cache hit too
            else:
                uncached_titles.append(title)

    # Cache misses: parallel fetch with as_completed
    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        future_to_title = {
            executor.submit(bound_fetch_fn, title): title
            for title in uncached_titles
        }
        for future in as_completed(future_to_title):
            title = future_to_title[future]
            page_result = future.result()   # raises WikipediaAPIError on failure
            results.update(page_result)
            if on_page_fetched:
                on_page_fetched(title, page_result.get(title, []))

    # Cache fresh results
    for title, links in fresh_results.items():
        self.cache_service.set(f"{cache_prefix}:{title}", links, ttl=self.cache_ttl)
    return results
```

Listing 5: wikipedia.py – _bulk_fetch firing on_page_fetched

Why `as_completed` instead of `executor.map`: `map` collects all results before yielding, so callbacks would bunch together at the end. `as_completed` yields each future the moment its HTTP response arrives — updates are spread across the 500–2000 ms fetch window, giving the polling client genuinely smooth feedback.

Why cache hits also fire the callback: If most of a batch is cached, the thread pool finishes almost instantly and the callback would never fire. Firing for cache hits ensures the client always gets progress updates even in warm-cache scenarios.

Thread safety: `on_page_fetched` is called concurrently from multiple worker threads. `nodes_lock` ensures `nodes_explored` is incremented atomically. `update_state` writes to Redis, which is inherently thread-safe.

Loop variable capture (B023): Python closures capture variables by reference. If `current_depth` were referenced as a free variable inside the closure, all threads would see whatever value it holds when they *execute* (the next batch's depth), not when the closure was *defined*. Binding it as a keyword-only default argument (`_d=current_depth`) forces evaluation at definition time, making each iteration's closure hold its own snapshot.

5.3.5 Step 4b: Link-Processing Callbacks Close the UI Gap

After `get_links_bulk` returns, the link-processing loop runs: for each of the 50 pages, it checks the visited set, stores new parent links in Redis, and pushes newly discovered nodes onto the queue. This loop is entirely Redis round-trips and can take several seconds for pages with hundreds of links. No Wikipedia fetch is in flight during this time, so without intervention the client would stall.

To prevent this, `progress_callback` is fired once per page *after* its links have been enqueued. At that point `queue_size` reflects the newly added nodes:

```
for item in batch_items:
    current_page = item["page"]
    links = links_bulk.get(current_page, [])

    for link in links:
        if link == end_page:
            # Store parent and reconstruct immediately
            self.cache_service.hash_set(parent_key, link, current_page)
            final_path = self._reconstruct_path(end_page, parent_key)
            return {"path": final_path, "nodes_explored": nodes_explored}

        if not self.cache_service.set_contains(visited_key, link):
            self.cache_service.set_add(visited_key, link)
            self.cache_service.hash_set(parent_key, link, current_page)
            self.queue_service.push(queue_key,
                {"page": link, "depth": item["depth"] + 1})

    # Fire after all links for this page are enqueued -- queue_size is now accurate.
    if self.progress_callback:
        self.progress_callback({
            "status": "Searching...",
            "search_stats": {
                "nodes_explored": nodes_explored,
                "current_depth": current_depth,
                "last_node": current_page,
                "queue_size": self.queue_service.length(queue_key),
            },
            "search_time_elapsed": round(time.time() - search_start_time, 2),
        })
```

Listing 6: pathfinding.py – Per-page callback during link processing

5.3.6 Path Reconstruction via Parent Hash

The BFS never stores explicit path lists. Instead, it records each discovered node's parent in a Redis hash:

- **Key:** `bfs_parent:<uuid>` (a single Redis hash per search)
- **Field:** child page title; **Value:** parent page title

At the end, `_reconstruct_path(end_page, parent_key)` walks the hash backwards until it reaches `start_page` (which has no parent entry), then reverses the list.

5.3.7 Step 5: Client Sees Progress via Polling

```
GET /tasks/status/abc-123

{
    "status": "IN_PROGRESS",
    "task_id": "abc-123",
    "progress": {
        "status": "Searching...",
        "search_stats": {
            "nodes_explored": 42,
            "current_depth": 3,
            "last_node": "Machine learning",
            "queue_size": 150,
            "start_page": "Python (programming language)",
            "end_page": "Alan Turing",
            "max_depth": 6
        },
        "search_time_elapsed": 5.2
    }
}
```

Listing 7: Client Polling Response

5.4 Bidirectional BFS Progress: BidirProgressAggregator

The bidirectional pathfinder runs two concurrent frontiers. `BidirProgressAggregator` sits between them and the single progress callback:

```
class BidirProgressAggregator:
    def __init__(self, callback, queue_service, fwd_q, bwd_q):
        self._callback = callback
        self._queue_service = queue_service
        self._fwd_q, self._bwd_q = fwd_q, bwd_q
        self._fwd_nodes = self._bwd_nodes = 0
        self._lock = threading.Lock()

    def record(self, title: str, depth: int, direction: str) -> None:
        with self._lock:
            if direction == "forward":
                self._fwd_nodes += 1
            else:
                self._bwd_nodes += 1
            combined_queue = (self._queue_service.length(self._fwd_q)
                              + self._queue_service.length(self._bwd_q))
            self._callback({
                "nodes_explored": self._fwd_nodes + self._bwd_nodes,
```

```
                "queue_size": combined_queue,
                "current_depth": depth,
                "last_node": title,
                "direction": direction,
            })

    @property
    def total_nodes(self) -> int:
        return self._fwd_nodes + self._bwd_nodes
```

Listing 8: pathfinding.py – BidirProgressAggregator

5.5 Why This Design?

Benefit	Explanation
Decoupling	The BFS algorithm doesn't know about Celery. It just calls a function.
Testability	In tests, pass a mock callback that records calls instead of updating Celery.
Flexibility	Easy to change reporting frequency or add more metrics without touching BFS.
No Polling Inside Worker	Worker doesn't poll; it pushes. Client pulls on its own schedule.

6 Dependency Injection (DI) Explained

Note

Location: `app/core/factory.py` – `ServiceFactory` class

6.1 The Pattern: Singleton + Factory

```
class ServiceFactory:
    _redis_client = None      # Singleton storage
    _cache_service = None
    _queue_service = None
    _wikipedia_client = None

    @classmethod
    def get_cache_service(cls):
        if cls._cache_service is None:
            redis_client = cls.get_redis_client()
            cache_ttl = current_app.config.get("CACHE_TTL", 86400)
            cls._cache_service = RedisCache(redis_client, cache_ttl)
        return cls._cache_service

    @classmethod
    def create_pathfinding_service(cls, algorithm="bidirectional",
   progress_callback=None):
        wiki    = cls.get_wikipedia_client()
        cache   = cls.get_cache_service()
        queue   = cls.get_queue_service()
        max_depth  = current_app.config.get("MAX_SEARCH_DEPTH", 6)
        batch_size = current_app.config.get("BFS_BATCH_SIZE", 50)

        if algorithm.lower() == "bidirectional":
            path_finder = BidirectionalBFSPathFinder(
                wiki, cache, queue, max_depth, batch_size, progress_callback)
```

```
        else:
            path_finder = RedisBasedBFSPathFinder(
                wiki, cache, queue, max_depth, batch_size, progress_callback)

        return PathFindingService(path_finder, cache, wiki)
```

Listing 9: ServiceFactory Implementation

6.2 Swapping Implementations

To use Memcached instead of Redis:

1. Create `MemcachedCache` implementing `CacheServiceInterface`.
2. Modify `ServiceFactory.get_cache_service()` to return `MemcachedCache`.
3. Done. All services automatically use the new cache.

7 File-by-File Breakdown

7.1 Core Files

File	Purpose
`app/core/pathfinding.py`	**CORE ALGORITHM**. `RedisBasedBFSPathFinder` (forward BFS) and `BidirectionalBFSPathFinder` (bidir BFS). Also `BidirProgressAggregator`.
`app/core/services.py`	Business orchestration. Cache checking, validation, timing. `PathFindingService`, `WikipediaService`, `CacheManagementService`.
`app/core/factory.py`	DI container. Manages singletons and object creation.
`app/core/interfaces.py`	Abstract contracts for testability and loose coupling.
`app/core/models.py`	Dataclasses: `PathResult`, `SearchRequest`, `WikipediaPage`, `TaskStatus`, `HealthStatus`, `CacheStats`.

7.2 Infrastructure Files

File	Purpose
`app/infrastructure/cache.py`	`RedisCache`: JSON-serialised get/set/delete, set operations (`set_add`, `set_contains`, `set_contains_many`, `set_add_many`), hash operations (`hash_set`, `hash_get`, `hash_set_many`), `delete_many`, `clear_pattern`. Pipeline used for batch ops.
`app/infrastructure/redis_queue.py`	`RedisQueue`: FIFO queue for BFS state (`push`, `pop`, `push_batch`, `pop_batch`, `length`, `clear`). Pipeline used for batch ops.
`app/infrastructure/tasks.py`	Celery task definitions: `find_path_task` (main), `health_check_task`, `cache_cleanup_task`. Retry logic (`autoretry_for` + manual retry with exponential backoff). Progress updates via `update_state`.
`app/external/wikipedia.py`	`WikipediaClient`: `get_links_bulk`, `get_backlinks_bulk`, `get_page_info`, `get_page_with_redirect_info`. Shared `_bulk_fetch` scaffold with cache, thread pool, `as_completed`. Per-request rate limiting and exponential backoff.

8 Caching Strategy

Key Pattern	TTL	Purpose
`wiki_links:{page}`	24h	Caches outgoing links for a Wikipedia page (`get_links_bulk`).
`wiki_backlinks:{page}`	24h	Caches backlinks ("what links here") for a page (`get_backlinks_bulk`).
`path:{start}:{end}`	1h	Caches completed path results to avoid re-searching.
`page_info:{page}`	2h	Caches page metadata (`WikipediaService.get_page_info`).
`bfs_queue:{uuid}`	Deleted	BFS frontier queue. Explicitly cleaned in `finally` block after search.
`bfs_visited:{uuid}`	Deleted	Redis set of visited page titles for the search session.
`bfs_parent:{uuid}`	Deleted	Redis hash mapping child → parent for path reconstruction.
`bfs_fwd_queue:{sid}`	Deleted	Forward queue for bidirectional BFS.
`bfs_bwd_queue:{sid}`	Deleted	Backward queue for bidirectional BFS.
`bfs_fwd_visited:{sid}`	Deleted	Forward visited set for bidirectional BFS.
`bfs_bwd_visited:{sid}`	Deleted	Backward visited set for bidirectional BFS.
`bfs_fwd_parent:{sid}`	Deleted	Forward parent hash for bidirectional BFS.
`bfs_bwd_parent:{sid}`	Deleted	Backward parent hash for bidirectional BFS.

Note

The `cache_cleanup_task` Celery Beat job sweeps `bfs_*` keys hourly to clean up any orphaned search state from crashed workers. Normal search completion deletes keys immediately in a `finally` block.

9 Testing Architecture

Note

All tests run with **no real Redis and no real Wikipedia API calls**. Everything external is mocked.

9.1 Directory Structure

File	What it tests
`tests/conftest.py`	Shared fixtures: app, client, all mock services.
`tests/unit/test_models.py`	Dataclass validation logic (`PathResult.is_valid`, `SearchRequest.validate`).
`tests/unit/test_exceptions.py`	Every custom exception class and its message format.
`tests/unit/test_cache_impl.py`	`RedisCache` against a mock Redis client: get/set/delete, set ops, hash ops, pipeline batching, error propagation.
`tests/unit/test_redis_queue_impl.py`	`RedisQueue` push/pop/batch/clear against mock Redis.
`tests/unit/test_external_wikipedia_client.py`	`WikipediaClient`: cache hits/misses, link filtering, redirect/disambiguation handling, backoff behaviour.
`tests/unit/test_factory_and_tasks_config.py`	`ServiceFactory` lifecycle (create, singleton reuse, cleanup). Celery task routing config.
`tests/unit/test_tasks.py`	All branches of `find_path_task`, `health_check_task`, `cache_cleanup_task` via Celery eager execution.
`tests/unit/test_middleware_and_logging.py`	All middleware decorators (error handling, CORS, rate limiting, request size).
`tests/integration/test_api_endpoints.py`	Flask routes end-to-end with mocked Celery tasks.
`tests/integration/test_services.py`	`PathFindingService`, `WikipediaService`, `CacheManagementService` with mocked dependencies.
`tests/integration/test_pathfinding.py`	BFS algorithms with stateful mock queue + cache. Also `BidirProgressAggregator`.

9.2 Mock Fixture Design

Fixture	Strategy
`mock_redis`	Raw `Mock()` with a hand-rolled `_MockPipeline` that supports `rpush/lpop/execute` and context manager protocol.
`mock_cache_service`	Stateful: backed by an in-memory `dict` (get/set), `dict[str,set]` (set ops), and `dict[str,dict]` (hash ops). Behaves identically to `RedisCache` from the caller's perspective.
`mock_queue_service`	Stateful: backed by an in-memory `dict[str, list]`. Supports push/pop/push_batch/pop_batch/length/clear.
`mock_wikipedia_client`	Simple `Mock()` with default return values. Tests override specific methods as needed.
`mock_path_finder`	`Mock()` with `find_path.return_value = {"path": [...], "nodes_explored": 10}`.
`mock_celery_task`	Context manager that patches `find_path_task` in `routes.py` namespace. Exposes `mock_async_result` for state simulation.
`celery_memory_backend`	Sets `task_always_eager=True` + `result_backend="cache+memory://"` for task tests. Resets after test.

9.3 Testing Celery Tasks

Tasks are exercised via `task.apply(args=[...]).result` which runs them synchronously in the current process. The `celery_memory_backend` fixture enables `task_always_eager=True` so `update_state` works without a live Redis connection.

Key patterns:

- **Happy path:** Patch `get_pathfinding_service` to return a mock that returns a `PathResult`. Assert `result["status"] == "SUCCESS"`.
- **Page not found:** Mock `validate_pages` to return `(False, True, {})`. Assert `result["code"] == "PAGE_NOT_FOUND"`.
- **Max retries:** Call `task.apply(args=[...], retries=3)` to set `self.request.retries == max_retries`. Assert `result["code"] == "MAX_RETRIES_EXCEEDED"`.
- **Progress callback:** Assert that the second argument passed to `get_pathfinding_service` is callable.

9.4 Coverage

Current overall coverage: **89%**. Key uncovered lines are in `app/__init__.py` (Celery integration initialisation) and some wikipedia.py pagination and error paths that require careful network simulation.

10 Interview Q&A Cheat Sheet

Q: If you had to modify the core pathfinding algorithm

Answer: `app/core/pathfinding.py` – specifically `RedisBasedBFSPathFinder._perform_bfs_search()` for forward BFS, or `BidirectionalBFSPathFinder._expand_forward_batch()` / `_expand_backward_batch()` for bidirectional BFS.

Q: When and where do services get initialized?

Answer: Services are **lazily initialized** by `ServiceFactory` (`app/core/factory.py`) on first use. Not at app startup. Flask: first request. Celery: first task.

Q: Which objects are reused vs. created fresh?

Answer:
- **Reused (Singletons):** Redis Client, Wikipedia Client, Cache Service, Queue Service. Maintains connection pools and rate-limiter state.
- **Fresh (Transient):** PathFindingService and its PathFinder. New per task for isolated callbacks and algorithm selection.

Q: How does the progress callback work?

Answer:
1. Celery task defines a closure that calls `self.update_state(state="PROGRESS", meta=...)`.
2. This closure is passed to `ServiceFactory.create_pathfinding_service()`.
3. Factory injects it into `RedisBasedBFSPathFinder` (or `BidirectionalBFSPathFinder`) constructor.
4. BFS defines an `on_page_fetched` closure that it passes to `get_links_bulk`.
5. Wikipedia client fires the closure for each page as its response arrives (via `as_completed`) *and* for cache hits.
6. A second callback fires after link processing per page (to report accurate queue size).
7. Client polls `/tasks/status/<id>` to read `state=PROGRESS` updates.

Q: Explain the two BFS algorithms and when each is used.

Answer:
- **bfs (RedisBasedBFSPathFinder):** Standard forward BFS. Expands from start page only, following outgoing links. Simple, predictable.
- **bidirectional (BidirectionalBFSPathFinder):** Two frontiers expanding simultaneously. Forward follows outgoing links (`get_links_bulk`); backward follows backlinks (`get_backlinks_bulk` / Wikipedia's "what links here"). Always expands the smaller frontier. Halts when a node appears in both visited sets. Path is forward_path + backward_chain stitched at the meeting node.
- **Default:** `bidirectional`. Caller selects via the `algorithm` field in `POST /getPath`.

Q: Why use Celery instead of handling pathfinding synchronously?

Answer: Pathfinding can take minutes. Synchronous handling would:
- Timeout HTTP connections (typically 30-60s limit).
- Block web server threads, reducing throughput.
- Provide no progress visibility to users.

Celery enables horizontal scaling, resilience, and real-time progress.

Q: Why Redis for BFS queue instead of Python deque?

Answer: Memory safety. BFS at depth 5+ can involve millions of nodes. Redis:
- Offloads memory from worker process.
- Enables inspection via Redis CLI.

- Provides crash-recovery potential (state persists in Redis if worker dies).

Trade-off: Network latency per operation (mitigated by batching with `pop_batch`).

Q: How does the system scale horizontally?

Answer:
- **API:** Stateless Flask servers behind load balancer.
- **Workers:** Add Celery workers to increase search throughput.
- **Cache:** Redis Cluster for sharding and replication.

Q: Explain singletons across processes.

Answer: Singletons are per-process. 4 Gunicorn workers = 4 Wikipedia Clients. They share state via Redis (the database is the shared memory), not in-process memory.

Q: Why is the Wikipedia Client a singleton?

Answer: It holds a `requests.Session` which provides:
- HTTP Keep-Alive (connection reuse).
- TLS session resumption (avoids 100-300ms handshake).
- A shared rate-limiter lock + timestamp (ensures all threads from this process respect the rate limit globally).

Creating new clients per request would destroy these benefits.

Q: How would you add a new pathfinding algorithm?

Answer:
1. Create class implementing `PathFinderInterface` in `pathfinding.py`.
2. Add an `elif` branch in `ServiceFactory.create_pathfinding_service()`.
3. Update `SearchRequestSchema` to accept the new algorithm name.
4. Add integration tests in `tests/integration/test_pathfinding.py`.

Q: How are tests structured? What's mocked?

Answer: Tests live in `tests/unit/` (per-class) and `tests/integration/` (service interactions). No real Redis or Wikipedia calls. The `mock_cache_service` and `mock_queue_service` fixtures in `conftest.py` are *stateful* in-memory mocks that behave identically to the real implementations. Celery tasks are tested via `task.apply()` in eager mode with an in-memory result backend.

11 Quick Reference: Where to Make Changes

Goal	Location
Modify forward BFS	`app/core/pathfinding.py` – `RedisBasedBFSPathFinder`
Modify bidirectional BFS	`app/core/pathfinding.py` – `BidirectionalBFSPathFinder`
Add new API endpoint	`app/api/routes.py` (add route + Flasgger YAML docstring)
Change request validation	`app/api/schemas.py`
Add new Celery task	`app/infrastructure/tasks.py`
Swap cache implementation	Implement `CacheServiceInterface`, update `factory.py`
Add new exception type	`app/utils/exceptions.py`
Change configuration	`config/base.py`
Modify Wikipedia API logic	`app/external/wikipedia.py`
Add tests	`tests/unit/` for isolated class tests, `tests/integration/` for service interactions

12 API Endpoints Reference

Method	Endpoint	Description
POST	`/getPath`	Start pathfinding. Returns task ID + poll URL (202 Accepted).
GET	`/tasks/status/<id>`	Poll for task status: PENDING, IN_PROGRESS (PROGRESS state), SUCCESS, FAILURE, RETRY.
GET	`/tasks`	List all active, reserved, and scheduled Celery tasks across workers. Returns counts and raw task metadata. 503 if no workers reachable.
DELETE	`/tasks/<id>`	Revoke (cancel) a specific task. Sends SIGTERM if already running (`?terminate=false` to skip). Returns 404 if task already in SUCCESS/FAILURE.
DELETE	`/tasks`	Revoke all active and reserved tasks across all workers. Accepts same `?terminate` query param.
GET	`/health`	System health check: Redis ping, cache round-trip, Wikipedia client init.
GET	`/`	Serve static web UI (`static/index.html`). All non-API paths redirect here.
GET	`/api`	API metadata and endpoint list.
GET	`/api/docs`	Swagger UI (auto-assembled from YAML docstrings via Flasgger).
POST	`/cache/clear`	Admin: clear Redis cache by key pattern (default: `wiki_links:*`).

Note

The `catch_all` route (`/<path:path>`) redirects all unrecognised paths to `/`. Paths starting with `getPath`, `tasks`, `health`, `cache`, or `api` return a structured JSON 404 instead.
Task management endpoints use `celery.control.inspect(timeout=2.0)` for listing and `celery.control.revoke()` for cancellation. The inspect call broadcasts to all workers and aggregates responses; a 2-second timeout avoids hanging the HTTP request when workers are unavailable.

13 Configuration Reference

Variable	Default	Description
`REDIS_URL`	redis://localhost:6379/0	Redis connection URL.
`MAX_SEARCH_DEPTH`	6	Maximum BFS depth before giving up.
`BFS_BATCH_SIZE`	50	Pages per batch popped from queue.
`CACHE_TTL`	86400	Default cache TTL for wiki_links (24h).
`WIKIPEDIA_MAX_WORKERS`	10	Thread pool size for parallel Wikipedia API calls.
`WIKIPEDIA_MAX_PAGINATE_CALLS`	3	Pagination calls per page ($3 \times 500 = 1500$ links max).
`WIKIPEDIA_REQUEST_DELAY`	0.1	Minimum seconds between Wikipedia requests (rate limit).
`WIKIPEDIA_BACKOFF_MAX_RETRIES`	5	Max retry attempts with exponential backoff.
`CELERY_TASK_SOFT_TIME_LIMIT`	300	Soft timeout for pathfinding task (5 min).
`CELERY_TASK_TIME_LIMIT`	600	Hard timeout for pathfinding task (10 min).
`LOG_LEVEL`	INFO	Logging level.